Embedded Firmware Design Assessment

Objective

Design the software architecture and develop the core pseudocode for an RTOS-based Flight Sequencer running on a multi-core processor. This assessment evaluates your ability to design deterministic real-time systems, manage inter-process communication (IPC), partition tasks across multiple cores, and protect critical data sections.

System Overview

You are tasked with designing a Flight Sequencer subsystem for a launch vehicle. The system interfaces with multiple navigation sensors and actuators via RS422 buses. The software runs on a multi-core microcontroller running a preemptive RTOS.

System Requirements

1. Sensor Interfaces (Inputs)

- **Navigation Sensor 1:** Receives a 32-byte payload via a dedicated RS422 bus at a strict periodicity of **5 ms**.
- **Navigation Sensor 2:** Receives a 54-byte payload via a second dedicated RS422 bus at a strict periodicity of **10 ms**.
- Both data streams are passed as inputs to the main Flight Sequencer computation.

2. Core Computation Tasks

- **Flight Sequencer Task (10 ms cycle):** Consumes the latest navigation data, performs necessary flight computations, and generates sequencing and actuation commands.
- **Slow Computation Task (200 ms cycle):** A secondary computation cycle that executes every 200 ms. It consumes the latest navigation inputs and a specific 16-byte output generated by the Flight Sequencer.

3. Actuation & Feedback (Outputs)

- **Actuator Output:** The 10 ms Flight Sequencer task outputs a 20-byte command payload forwarded to an actuator via a third RS422 port.
- **Algorithm Feedback Loop:** The 200 ms computation cycle produces a 16-byte output that must be fed back into the 10 ms Flight Sequencer cycle for its next iteration.

4. Telemetry & Health Monitoring

- **Supervisory Task (10 ms cycle):** Monitors and telemeters the health data of the sensor interfaces, actuator interfaces, Flight Sequencer task, and 200 ms computation task. This health data is transmitted via a fourth RS422 port.

Candidate Deliverables

Note on Assumptions: Open-ended system designs often lack specific hardware details (e.g., DMA capabilities, specific RTOS overhead, FIFO buffer sizes). If you determine that critical information is missing to complete your architecture, you are expected to make reasonable engineering assumptions. **Please explicitly document all assumptions at the beginning of your submitted report.**

Please provide a comprehensive document and pseudocode addressing the following:

Part 1: System Architecture & Task Design

1. **Task Definition:** List all RTOS tasks, timers, or ISRs required for this system. For each, define its priority, periodicity, and core assignment (Core 0, Core 1, etc.).
2. **Multi-Core Strategy:** Explain your rationale for mapping specific tasks to specific cores. How does your design balance the load and minimize contention?
3. **Data Flow & IPC:** Map out the Inter-Process Communication (IPC) mechanisms (e.g., queues, mutexes, semaphores, shared memory) used to pass data between the 5ms/10ms inputs, the 10ms sequencer, the 200ms task, and the supervisory task.
4. **Hardware Interface Strategy:** Explicitly weigh the **pros and cons of using interrupts vs. polling** for the four RS422 interfaces in the context of this specific timing budget. State which method you chose for each interface and why.

Part 2: Synchronization & Concurrency

1. **Critical Sections:** Identify the shared resources and critical sections in this architecture (e.g., the 16-byte feedback loop, the latest navigation data).
2. **Data Protection:** Explain how your design ensures data integrity and prevents race conditions, deadlocks, or priority inversion, especially when data is shared across different cores and task frequencies.

Part 3: Pseudocode Implementation

Provide C-style pseudocode using generic RTOS APIs (e.g., FreeRTOS, POSIX, or CMSIS-RTOS standard functions) for the following:

- The system initialization block (task creation, IPC primitive creation, core affinity setup).
- The peripheral data reception mechanism (Interrupt handlers or polling loops for Sensor 1 and Sensor 2).
- The **10 ms Flight Sequencer Task** (showing data ingestion, the feedback loop integration, and the command transmission).
- The **200 ms Computation Task**.
- The **10 ms Supervisory Task**.

(Note: You do not need to write the actual flight math; simply use placeholder functions like ComputeFlightActuation() or CalculateSlowDynamics() where appropriate, focusing entirely on the structural timing, RTOS primitives, and data flow).

Evaluation Criteria

- **Real-Time Determinism:** Does the architecture guarantee that the 5ms and 10ms deadlines will not be missed?
- **RTOS Fundamentals:** Proper use of IPC (queues vs. semaphores vs. mutexes) and understanding of task blocking, unblocking, and scheduling.
- **Multi-Core Competency:** Logical partitioning of tasks and safe data sharing across a multi-core boundary.
- **Engineering Judgment:** The quality, realism, and documentation of any assumptions made to bridge gaps in the requirements.
- **Design Justification:** Depth of technical reasoning, particularly in the interrupts vs. polling analysis and synchronization strategies.